

AI for Economic Research: Dynamic Models, Language, and Agents
Lecture 9.2b: Anatomy of an Agent — Sessions, Sub-Agents, and Skills

Zhigang Feng

2026-09-12 12:10:31

Topic 9.2b Agenda

T2 opened the hood on one agent. This interlude answers the six questions students actually ask once they have run a session — and corrects three beliefs that are widespread and wrong.

1. **One agent, written out** — what a “session” physically is, and the loop in twenty lines of Python
2. **Three backgrounds, three agents** — the eight steps that turn a job description into a file you can dispatch
3. **Many agents** — how you create a *set* of them, and what actually limits the number
4. **Who decides to dispatch?** — the router myth, and what really happens
5. **Agent vs. skill** — a distinction most write-ups get backwards
6. **Synthesis** — and a lab where you design and dispatch your own agents

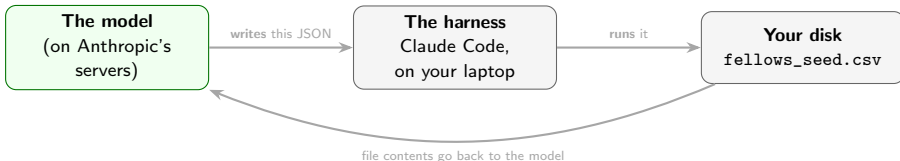
Why a separate deck. T1b surveyed the *landscape* of multi-agent systems; T4 taught you to *use* sub-agents. Neither showed the machinery, and neither had you *build* one. Everything here is checkable on your own machine — and the lab makes you check it.

One Agent, Written Out

What a session is, and the loop that drives it

From One JSON Line to a Running Loop

When an agent wants to read a file, it does not read the file. It writes a request — one line of JSON — and something else does the reading.



What the model emits (T2's line):

```
{"tool": "Read",  
  "args": {"file_path": "..."}}
```

It is *text the model generated*, in a shape you told it to use. The model cannot touch your disk; the harness can.

The naive mental model. “I open a session, the model remembers me, and somewhere a dispatcher assigns work to specialists.”

Every clause is wrong. This deck replaces it — and the cost model, the context limits, and the sub-agent behaviour of T4–T5 all follow.

Answered next: if nothing is stored, what is a “session” — and where is “the agent”?

First, the Vocabulary: Who Is Speaking?

Every message carries a `role` label saying who produced it. There are three things you will see, and one is misleadingly named.

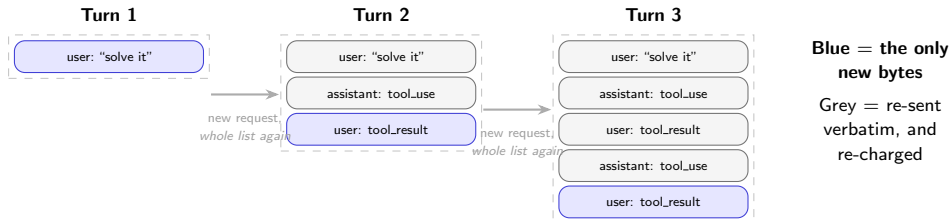
Label	Who actually wrote it	Example
"user"	you	"Solve the model and report r ."
"assistant"	the LLM — Claude itself	the JSON tool request on the last slide
"user", containing a <code>tool_result</code>	your harness, reporting what a tool returned	the contents of <code>fellows_seed.csv</code>

The naming trap. "Assistant" is simply the API's word for *the model's own turn* — it is not a separate helper program. And tool results are labelled "user" even though you did not type them: the API has only two speakers, so anything sent to the model is labelled "user", whoever produced it. Read "user" as "input to the model", not as "the human".

Why it matters: assistant means *the model*; tool output re-enters as user. That is all the vocabulary this deck needs.

The Session Is a List, Not a Connection

The API is stateless. Each turn is a separate request that carries the entire conversation so far — the model remembers nothing between calls.



- **“Re-send everything” is literal:** turn 3 is a fresh request carrying turns 1–3 in full
- No open connection, no server-side memory of you
- “Context window full” means *this list* outgrew the limit
- You are billed for the whole list each turn

Where the list starts and stops. /clear does exactly one thing: it throws the list away and starts an empty one. Compaction replaces the list with a summary of itself. Both are list operations — there is no “connection” to reset.

Four Moving Parts, One Function Call

An agent is not a service you connect to. It is four arguments you assemble and send.

Argument	What it is	Concretely
model	which LLM to send to	the string "claude-opus-5" — just a name; you are picking a brain
system	the standing instructions : role, rules, style, read <i>before</i> the conversation	"You are a computational macroeconomist. Never invent data." This is what CLAUDE.md becomes (T4)
tools	the menu of actions — name, description, argument schema. <i>Descriptions only; no code is sent</i>	"Read: read a file. Arguments: file_path (string)." The model picks from this menu
messages	the conversation so far	every user / assistant / tool_result turn, in order

What comes back: ordinary text, or tool_use blocks — the model asking you to run something from the menu.

Change system and tools and you have a *different agent*; there is no "agent object" to create or shut down.

The Loop in Twenty Lines — Written by You, Once

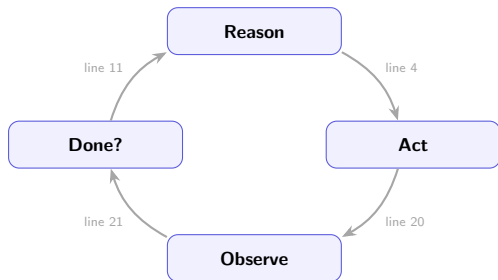
You write this loop once. The model never writes or runs it — it only supplies the replies the loop reads.

```
1 messages = [{"role": "user", "content": "Solve the model and report r."}]
2
3 while True:
4     resp = client.messages.create(
5         model="claude-opus-5",
6         max_tokens=16000,
7         system="You are a computational macroeconomist.",
8         tools=TOOLS,                # schemas, not functions
9         messages=messages,          # the whole list, every time
10    )
11    if resp.stop_reason != "tool_use":    # model is done talking
12        break
13    messages.append({"role": "assistant", "content": resp.content})
14
15    results = []
16    for b in resp.content:                # may be SEVERAL tool calls
17        if b.type == "tool_use":
18            results.append({"type": "tool_result",
19                            "tool_use_id": b.id,    # must match
20                            "content": run_tool(b.name, b.input)})
21    messages.append({"role": "user", "content": results})
```

Not automatic: this file — a person wrote it. **Automatic:** *which* tool, *which* arguments, when to stop — chosen by the model inside `create()`, line 4. **Line 9 with line 21:** the list only grows and is re-sent whole every pass.

ReAct, Named — and Where the Textbook Diagram Is Now Dated

The loop you just read has a name in the literature: **ReAct** — reason, act, observe, repeat (Yao et al., 2022).

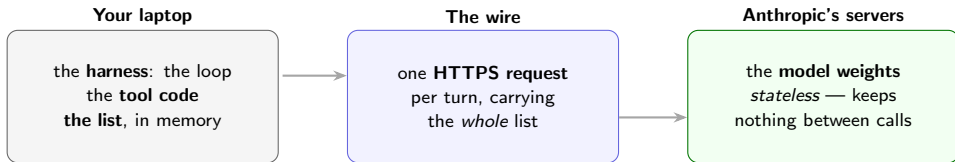


- **Reason** = the model's turn (line 4)
- **Act** = your harness runs the tool (line 20)
- **Observe** = the `tool_result` goes back (line 21)
- **Done?** = the `stop_reason` test (line 11)

Where the 2022 picture is dated. The original ReAct prompted the model to emit literal `Thought: / Action:` text that a parser scraped. Current models emit *structured* `tool_use` blocks and interleave reasoning with tool calls rather than strictly alternating. **Treat ReAct as the ancestor of the loop, not as its wire format** — if you implement the 2022 string-parsing version today you are rebuilding something the API already gives you typed.

So Where Does the Agent Live?

T2 wrote: “the agent reads it and decides the next action.” That is convenient shorthand. Taken literally it is misleading — no single place contains the agent.



The agent is the loop, not a location. “The agent” names a *behaviour* that appears when a stateless model is driven by a stateful loop. Delete the loop and there is no agent — only a model that answers one question. Read T2’s sentence as: *the harness appended the tool result to the list, and the next request carried that list to the model.*

Why this is the practical frame: it tells you where each cost and limit lives — tokens and the context window on the *wire and the server*, tool permissions and your data on *your laptop* (T2’s privacy boundary), and the memory in a list *you* own and can clear.

Three Backgrounds, Three Agents

Why the capstone needs all three — and the eight steps that build one

The Capstone Needs Three Backgrounds. The Syllabus Requires One.

The course assumes you are an economist. The capstone assumes you are also a programmer and a mathematician — and those two are exactly what it never asks for.

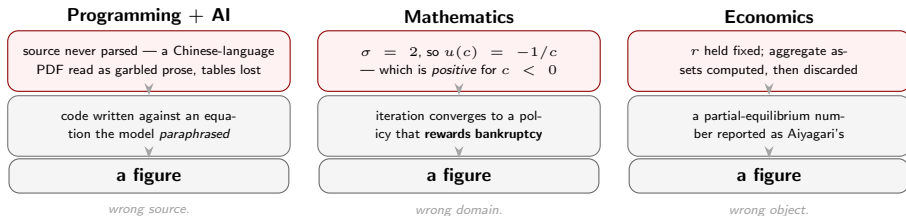
Background	What M3 “replication verified” needs	What the course assumes
Economics	an estimand or equilibrium concept, and a calibration you can defend	required — “intermediate micro and macro”
Mathematics	the domain of $u(\cdot)$, the optimality condition, the contraction, a fine enough grid	not required — recursive methods are “a plus”
Programming + AI	readable sources, a solver that runs, pinned environment, one-command reproduction	not required — “Python experience helps but is not required”

The gap is the assignment. The syllabus is not at fault — the course builds up what it needs. The problem is *sequencing*: on the day the capstone starts you are short two of the three, and the deliverables do not wait.

Answered next: what goes wrong when a background is missing — and why the output looks fine either way.

Three Error Channels — and All Three Render a Figure

A missing background does not announce itself. Each of the three produces a different error, in a different place — and all three produce a program that exits 0 and draws a clean figure.



Why three, and not one careful generalist. The three errors live in different places and are caught by different *reading*: a parse failure by counting words, a domain error by asking where a function is defined, a missing market-clearing condition by knowing what the paper's contribution was. **No single pass finds all three.**

Answered next: three people, or three job descriptions?

Hire Three Students, or Write Three Job Descriptions

The capstone allows a team of two to four with mixed skill sets, and says solo is entirely feasible. What a team supplies in people, a solo student supplies in files.

	Three students	Three agents, solo
Written at M1	a division of labour	three files in <code>.claude/agents/</code>
Fixed cost	recruiting, scheduling, trust	one afternoon of careful English
Marginal cost	their time, and yours	tokens, and <i>your</i> coordination cost (§3)
Independent judgement	three priors, three literatures read	executorial only — all three inherit your framing (T5)
Still entirely yours	the question, the validation, the defence	the question, the validation, the defence

A complement, not a substitute. A mathematician's job description does not give you a mathematician's judgement — it gives you a second reading by something instructed, in writing, to look only for mathematics. T5 makes the argument; the rule is: if you have a co-author who knows the math, use the co-author **and** the agent.

The Trio, Sorted by Which Background Supplies the Expertise

T4 sorted your RA team by review function. Sort the same folder by which human background supplies the expertise and a different — emptier — picture appears.

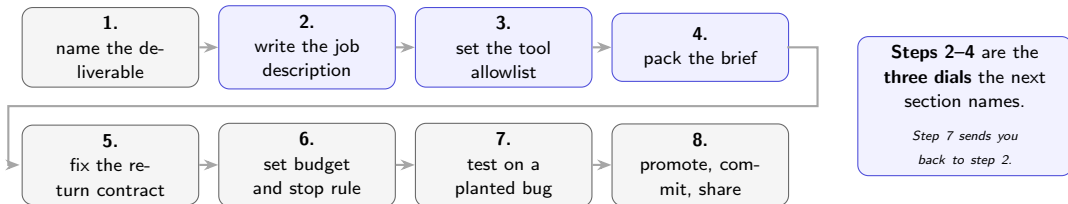
tooling-agent	math-agent	econ-agent
Programming + AI. Sources into context, models into runnable code: PDF to Markdown to $\text{\LaTeX}$, pinned environment, one-command reproduction. <i>Read, Bash, Glob, Write</i>	Mathematics. The derivation and the method: domain of $u(\cdot)$, the optimality conditions, the contraction, the grid and the tolerance. <i>Read, Grep, Glob</i>	Economics. The <i>object</i> : estimand or equilibrium concept, market clearing, calibration frequency, what the literature knows. <i>Read, Grep, Glob</i>

Now read the lab's own team again, and find the hole. code-reviewer checks the Python; domain-reviewer checks the code against the *stated* model; verifier runs it. **Nobody checks the stated model against the correct one.** The instructor key proves it: “utility() divides by $1 - \sigma$, zero in the log case” is filed under domain-reviewer — an *economist* auditing a limiting case, because no file owns that job. **[solid]**

T4 sorts these files by when they load; this section sorts them by which background wrote them. Only the second question tells you what is missing — and it is the mathematics agent you are going to write.

Eight Steps, Eight Artifacts — You Are Building an Agent, Not a Model

T3's six-step ladder builds a model. This ladder builds the thing that reviews it. Same discipline: every step ends in an artifact.



Eight artifacts, in order. A sentence of the form “*it produces X; I will know it failed when Y*”; a system prompt; a tools: line; a task string; a return format; a token ceiling and a stop rule; a 3×3 detection matrix; a committed file. **Seven of the eight produce English, not code — and a step that produced no artifact was skipped.**

Steps 1–2 — Name the Failure, Then Write the Job Description

Step 1 is one sentence: it produces X, and I will know it failed when Y. **Step 2** is that sentence expanded — the only part of an agent that is not a setting.

```
---
name: code-reviewer
description: Reviews Python numerical code for
  bugs, convergence problems, and silent failure
  modes. Use when the user asks for a code review
  or reports that results look wrong. Does NOT
  judge economics -- domain-reviewer's job.
tools: Read, Grep, Glob
model: sonnet
---
```

You are a careful reviewer of scientific Python.
This is research code, not production software:
correctness and silent failure matter far more than
style. Report, in this order:

1. Outright bugs -- wrong index, wrong sign,
wrong order of operations.
2. Silent failures -- a loop that exits on
iteration count rather than a convergence
criterion; a tolerance never checked.
3. Numerical fragility -- division without a
guard; a grid too coarse for the claim.

For each finding give the line number, quote the
line, and say what goes wrong at run time. If you
find nothing in a category, say so explicitly
rather than inventing an issue.

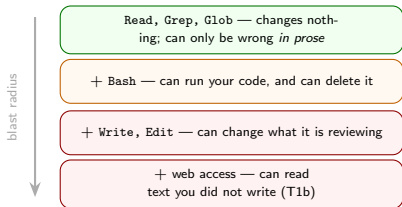
1. **Step 1 is the last clause of description:**
“Does NOT judge economics.” The failure mode
is named *before* the agent exists.
2. **Step 2 is everything below the second rule.**
That prose becomes the system argument of
every request this agent makes — §1's second
moving part, written by you.
3. **“say so explicitly rather than inventing an
issue”** is the most valuable sentence in the file.
An agent asked to review *will* find something.

Verbatim from this course's own lab,

*labs/Lec09_Agent_Lab/.claude/agents/. Every slide so far
has talked about these files; this is the first one you have
seen. [solid]*

Steps 3–4 — Capability Is Blast Radius; the Brief Is the Context

The tools: line is not a convenience setting. Read forwards it is everything this agent can do to your project; read backwards it is everything it is able to learn.



Step 4 — the brief. This is what enters messages, and nothing else does:

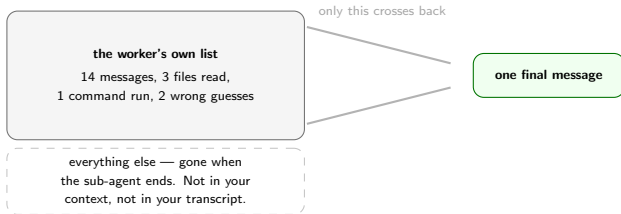
“Review `src/aiyagari.py` against `docs/model.md`. The calibration is annual. Report only findings you can tie to a line number.”

What stays out: your hypothesis, your hunch about where the bug is, and the dead ends you already tried. A clean list is the entire point — §3 draws it.

The asymmetry that makes the three non-interchangeable. `math-agent` needs *no* shell and *no* write access — a derivation is checked by reading, and an agent that cannot execute can never confuse “it ran” with “it is right.” tooling-agent is useless without Bash. Three dials, set three ways, are three different amounts of trust.

Steps 5–6 — One Message Back, and a Ceiling You Set

A sub-agent returns exactly one thing: its final message. Everything it read, ran, and reasoned about is discarded — so what you did not ask for, you will never see.



Step 5 — the return contract

- Fix the *shape*: one line per finding, giving `file:line`, the quoted text, and what breaks at run time
- Demand an explicit “nothing found in this category” — the sentence that stops an agreeable reviewer inventing one

Step 6 — budget and stop rule

- The `model`: line is a *budget* decision — a read-only reviewer on a 400-line file does not need the frontier model
- Stop rule: *report, do not fix*. A reviewer with Edit becomes an author, and you have lost your control group

Rate limits and context growth are §3's subject. Here: you set the ceiling, per agent, in one header line.

Steps 7–8 — Test It on a Bug You Planted, Then Commit It

An untested reviewer is one you trust for no reason. Plant one defect per background, dispatch all three, and read the matrix.

Defect (or control) in the solver	tooling-agent	math-agent	econ-agent
TOL defined, never read — loop exits on MAX_ITER	✓ <i>it finished, it did not converge</i>	✓ its job — no stopping rule	—
$\sigma = 2$, so $u(c) = -1/c$, <i>positive</i> for $c < 0$	✓ <i>seen in a run</i>	✓ its job	—
r fixed; aggregate assets computed, then <i>discarded</i>	—	—	✓ its job
<i>control</i> : $EV = P @ V.T$, which is correct	silent	silent	silent

Why the planted bug is a mathematics bug. With $\sigma = 2$, $utility(c) = c^{-1}/(-1) = -1/c$: negative for feasible $c > 0$, **positive** for infeasible $c < 0$. Infeasible therefore scores *higher* — +9.49 at $c = -0.13$ against -2.00 at $c = 0.50$ — and the program prints $V[0,0] = 9.490446$, then exits 0. **[solid]**

Read the matrix, not the reports. An empty column means the description is too narrow; a full column, too broad.

Step 8: commit the file — a teammate then gets your mathematician by `git pull`.

tooling-agent at Work: A PDF Becomes Readable Context

An agent cannot read a PDF — it reads text. Turning one into the other has a cost structure an economist should recognise.

```
export LC_ALL=en_US.UTF-8    # CJK paths, spaces

# stage every not-yet-converted PDF, then call
# the converter ONCE over the whole directory
STAGE="$(mktemp -d)"
for pdf in "$INPUT_DIR"/*.pdf; do
    name="$(basename "$pdf" .pdf)"
    [ -d "$OUTPUT_DIR/$name" ] && continue
    ln -s "$(readlink -f "$pdf")" "$STAGE/"
done

mineru -p "$STAGE" -o "$OUTPUT_DIR" \
    -b vlm-auto-engine

ls "$OUTPUT_DIR" | wc -l      # files kept?
wc -w "$OUTPUT_DIR"/*/vlm/*.md # words kept?
```

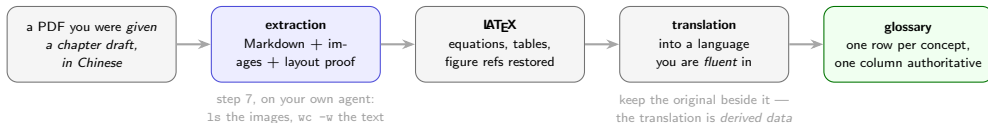
54-page econ paper, one A100-40GB	
engine startup + model load	1 m 50 s <i>fixed</i>
inference, all 54 pages	44 s <i>marginal</i>
total wall clock	2 m 59 s

Fixed cost 110 s; marginal cost 0.8 s a page. Calling the converter once *per file* re-pays the fixed cost every time: thirty PDFs become fifty-five minutes of model loading around twenty minutes of work. **One line of bash, and a returns-to-scale argument.**

Production code, NCSA Delta, via the scheduler — T4's login-versus-compute boundary. The silent failure in the same logs: one converter's concurrency default deadlocks the engine, which stalls while the server stays bound. Nothing errors. [solid]

From Markdown to a Language You Read — and a Stable Glossary

Extraction gets you text. It does not get you a source you can cite, or a term you can use twice the same way. Those are two more artifacts.



One glossary, extended — not a new one. This lecture's Chinese script already carries a 28-row bilingual key-terms table: *harness*, *orchestrator*, *sub-agent*, *context window*, *provenance*, *CLAUDE.md*, *skill*, *hook*, *compaction*, and twenty more. Add the rows your project needs and keep one column authoritative, so the same object is not called three things in three chapters. **Style is shared; vocabulary is local.**

Why this is not busywork. A term that drifts is a variable that drifts — and the glossary is what lets *math-agent* and *econ-agent* disagree about *economics* rather than about what you meant.

The Agent Design Canvas: Eight Cells, One Worked Answer

Eight steps, one page, filled in before you write a line. Bold is the question; grey is one worked answer; the file beside it is what that answer compiles to.

1. Produces / fails when <i>a math audit; fails by passing a wrong derivation</i>	2. Job description <i>mathematics — not code, not economics</i>
3. Tools <i>Read, Grep, Glob — no shell, no write</i>	4. Brief in <i>the solver, the model note, the frequency</i>
5. Returns <i>file:line, the algebra, "correct" if correct</i>	6. Budget / stop <i>one pass; report, do not fix</i>
7. Planted bug to catch <i>$u(c) = -1/c$ is positive for $c < 0$</i>	8. Path in the repo <i>.claude/agents/math-agent.md</i>

```
---
name: math-agent
description: Checks the mathematics of a numerical economics model -- the
  domain of every function evaluated, the derivation of the optimality
  conditions, and the convergence argument. Use when results are plausible
  but the model may be misstated. Does NOT review Python style and does NOT
  judge calibration -- econ-agent's job.
tools: Read, Grep, Glob
model: sonnet
---

You check mathematics. Assume the code runs and assume the economics is
someone else's job.

For every function the model evaluates, state its domain, check that the
code can only evaluate it inside that domain, and name the parameter value
that breaks it. Then: does the optimality condition follow from the stated
problem; is the operator a contraction, and is the stopping rule the one
that argument requires; is the grid fine enough for the claim being made?

Quote the line, give the algebra, and say plainly when a step is correct.
Never assert a result you cannot derive in the reply.
```

Then debug the English: ask it one thing it *should* handle and one it should *not*, and narrow the description until both behave (§4). **Blank canvas:** in the lab folder.

Bridge: Steps 2–4 Are the Three Dials

You have just written three job descriptions. The next section tells you what you actually did — you turned three dials on one function call, three times.



§1's four arguments. The fourth, `model`, is step 6 — your budget dial.

What carries forward. §3 names these dials and asks how many agents you can run at once, and what binds. §4 asks who decides to call them. §5 asks how a skill differs from an agent. **T2c asks what you actually type** — and what a run with ten thousand of them consists of. Every one is now a question about a Markdown file *you* wrote.

And the milestone, plainly: M1 asks for a division of labour. Working solo, `.claude/agents/` *is* your division of labour.

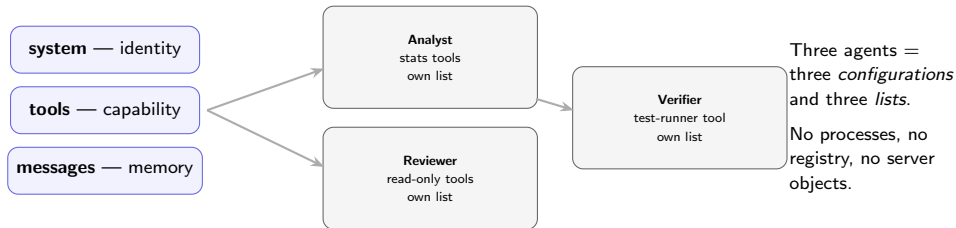
Answered next: how many can run at once, and what actually stops you.

Many Agents

Creating a set of them — and what actually limits the number

Three Dials Make an Agent

To create a set of agents you do not create anything. You vary three dials and keep three separate lists.



The economist's reading. You are not hiring staff; you are writing three job descriptions and keeping three notebooks. The “firm” exists only in your code. What is scarce is not agents — it is tokens, context, and your attention.

How Many at Once? Three Constraints, Not Infinity

“You can instantiate an unlimited number of agents” is true in the same sense that you can write an unlimited number of cheques.

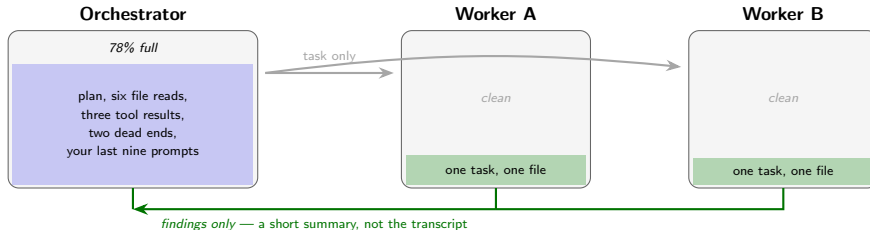


- **Rate limits** are the hard ceiling: quota is tokens-per-minute, not agents. Ten agents each re-sending a long list exhaust it faster than two.
- **Context** binds per agent: every worker carries its own growing list (previous section).
- **Coordination** is the one economists should expect: *you* must read, reconcile, and verify n outputs. That cost rises with n while the value of the marginal agent falls.

The binding constraint is usually the third one. T1b priced this out: multi-agent bought a large accuracy gain at roughly $15\times$ the token cost. Reach for more agents when the task is genuinely wide, not to feel parallel.

Isolated Context Is the Point

The reason to spawn a sub-agent is not extra brains — the model is the same. It is a second, clean list.

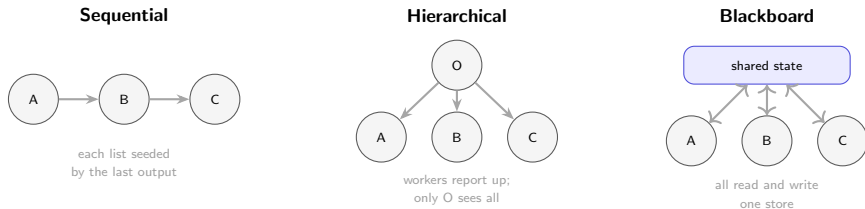


- The worker never sees your dead ends, so cannot be misled by them
- The orchestrator's list grows by a summary, not by all the worker read
- That asymmetry is the economic case for sub-agents

Do not over-read this. A clean window is not an independent mind. T5 shows workers inherit your framing — fresh context buys *focus*, not *independence*.

Three Wirings, Drawn as Data Flow

Strip the vocabulary away and multi-agent designs differ in exactly one respect: who writes into whose list.



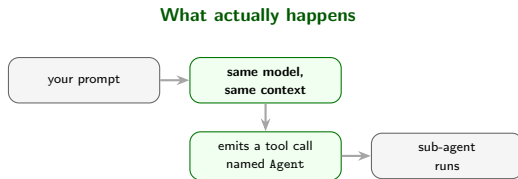
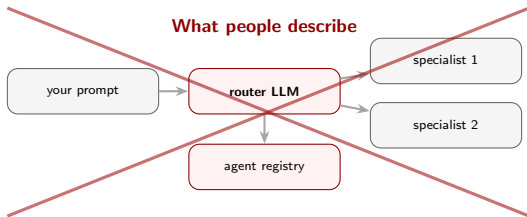
- **Sequential** is a relay: cheap, easy to debug, no parallel gain, and an early error propagates
- **Hierarchical** is T1b's orchestrator-worker and what T4's RA team already is — the pattern that dominates in practice
- **Blackboard** decouples agents from each other but needs write discipline; in a research project the "blackboard" is usually just *files on disk* — exactly the file-driven workflow of T4

Who Decides to Dispatch?

The router myth, and what the transcript actually shows

The Router Myth

The usual explanation: a fast “router” model classifies your prompt, looks up specialists in an agent registry, and wakes them. That is not what happens.



- **No second model, no classification step.** Dispatch is one more `tool_use` block from the §1 loop — the tool just happens to start another loop
- The “registry” is a folder of Markdown files *you wrote*; the model picks from it by reading their description fields, exactly as it picks any tool
- T2 said this for one agent: *“the same transformer that writes code decides to call tools.”* This is that fact one level up

Guard: T1b’s “think-budget routers” route one query between fast and deep modes of one model, not between agents.

Dispatch Is a Tool Call — Evidence You Can See

You do not have to take this on faith. Four checks, all runnable in the lab that follows.

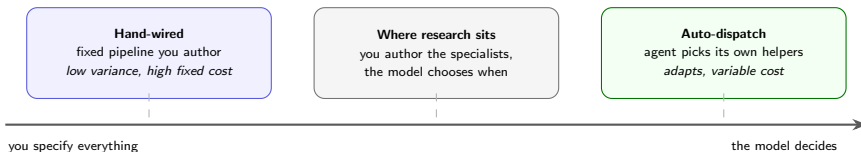
	Do this	What it proves
1	Run <code>/agents</code> in a session	The list is <i>your</i> <code>.claude/agents/</code> folder. Nothing is registered with a service.
2	Ask something a specialist fits; watch the transcript	The dispatch appears as a named tool call in the same stream as <code>Read</code> and <code>Bash</code> .
3	Compare context before and after	It grows by a short <i>summary</i> , not by the worker's transcript.
4	Narrow one agent's description, re-ask	Dispatch stops. Selection is driven by prose the model reads.

Check 4 is the decisive one. If a separate router with a registry were making the decision, editing one English sentence in a Markdown file could not switch the behaviour off. It can — because that sentence *is* the mechanism.

Real trace, check 2 (Claude Code 2.1.261): `Read` → *Agent* `code-reviewer` → *Agent* `domain-reviewer` — the dispatch sits in the same list as `Read`. **[solid]**

Auto-Dispatch vs. Hand-Wired: Spontaneous Order or Central Planning

Once you know dispatch is a model decision, the design choice becomes familiar: leave allocation to the agent, or plan it yourself.



Central planning (hand-wired)

- Reproducible: same path every run
- No planner tokens spent
- Brittle off the anticipated path
- High authoring cost; it decays

Spontaneous order (auto-dispatch)

- Handles tasks you did not foresee
- Zero setup at the point of use
- **Principal-agent problem:** it may delegate the wrong thing, and you pay
- Variable, harder-to-forecast cost

The practical middle. Author the specialists once, let the model choose *when* to call them, and keep a validation gate so a bad delegation is caught rather than trusted — T4's quality gates, doing an economist's job.

Agent vs. Skill

A distinction most write-ups get backwards

The Usual Definition Describes a Tool, Not a Skill

You will read that “a skill is a deterministic, hard-coded piece of software with no brain of its own.” That is an accurate definition — of something else.

The common claim

“A skill does exactly one thing. It requires exact inputs to produce exact outputs. It cannot decide when it should be used, nor fix itself on error. It just runs the code it was given.”

That paragraph is a correct description of a **tool** (a function with a schema). It is not what a skill is.

What a skill actually is

A folder with a Markdown file in it. The file is **prose written for the model to read** — a procedure, conventions, worked steps — plus optional scripts.

There is no compiled artefact and no fixed input schema. The reasoning is still the model's; the skill supplies *instructions*, not *execution*.

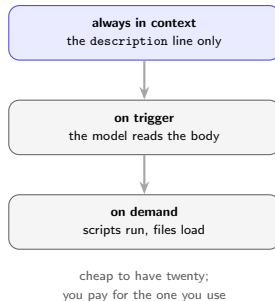
Why the confusion matters: if you believe a skill is code, you will try to write it like code — rigid parameters, defensive branching. Skills that work read like a good methods appendix written for a capable RA.

What a Skill Actually Is: Progressive Disclosure

The mechanism is a two-stage load, and it is the reason skills are cheap to keep around.

```
---
name: olg-5step
description: Manually run the local
  five-step OLG dynamic-macro demo.
  Use when explicitly invoked to
  demonstrate Model -> Equilibrium ->
  Algorithm -> Pseudo-code -> Implement.
disable-model-invocation: true
---

# OLG Five-Step Skill
Use this skill to demonstrate the five-
stage agentic dynamic-macro workflow.
## Workflow
1. Work from the demo root, the folder
   containing run.py, stages/, src/.
...
```



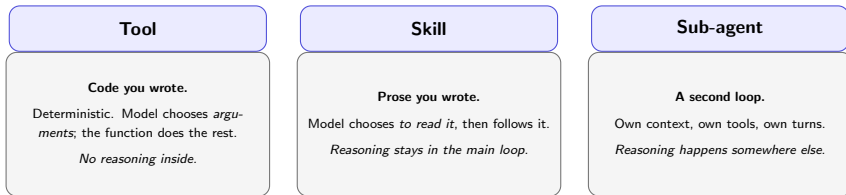
This repository's own skill:

demos/M4_olg_5step_demo/.claude/skills/olg-5step/SKILL.md

The description is load-bearing. It is the only part the model always sees, so it is what selection runs on — the same prose-driven choice as sub-agent dispatch in §4. T4 taught you *when* skills load; this is *how* they are chosen.

Tool, Skill, Sub-Agent: Sorted by Where the Reasoning Lives

T4 sorted Skills, Agents and Rules by when they load. Sort the same objects by who does the thinking and the confusion disappears.



	Tool	Skill	Sub-agent
Made of	code	Markdown	a task description
Costs you	a tool schema	one description line	a whole second context
Fails by	raising an error	being ignored	drifting off-task

T4's third category, Rules, is orthogonal to all three: it is always-on context, not a thing the model chooses.

disable-model-invocation: true — the **One-Line Proof**

One line in this repository's own skill settles the router question.

```
disable-model-invocation: true
```

- The flag means: *do not let the model decide to invoke this on its own* — it becomes callable by name only
- That flag is only meaningful because, by default, **the model itself** was making the call, by reading the description
- If a separate router service with a registry were dispatching, there would be nothing for a line of YAML inside one skill file to disable

The whole of §4 and §5 in one sentence. Selection — of a tool, of a skill, of a sub-agent — is the model reading short English descriptions and choosing. That is why editing prose changes behaviour, and why writing good descriptions is the highest-leverage thing you do when you set up a project.

A Worked Example, End to End

One real research question, traced message by message

The Task — and the Trap Inside It

One prompt, typed once:

“Get US real GDP growth since 2000. Get which party held the White House over the same period. Then plot and rationalize how party affiliation affects GDP growth.”

Three jobs, not one

1. fetch a macro series (*a fact*)
2. build a party timeline (*a fact*)
3. plot it (*mechanical*)
4. explain the relationship (*a causal claim*)

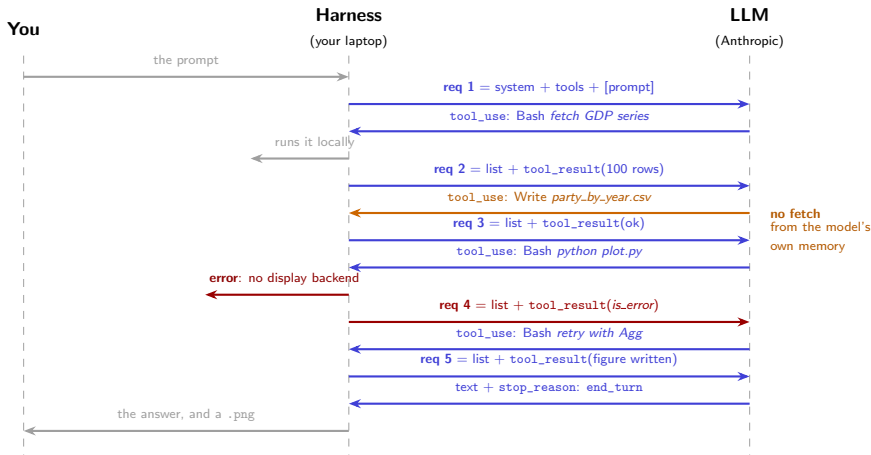
Steps 1–3 an agent can do. Step 4 is where the research is — and where the prompt is booby-trapped.

“Rationalize” is a leading verb. It presupposes an effect and asks for a justification. A compliant model will supply one. Watch, in the trace that follows, the exact turn where a defensible data task turns into an indefensible causal claim — and notice that nothing in the machinery stops it.

A chat model with no tools fails this instantly and invisibly: it will recite GDP numbers from memory. They will look right. Some will be wrong, and none will be checkable.

One Agent: Every Message That Crosses the Wire

Five requests. Each carries the entire list built so far.



Blue crosses the network and is billed; grey stays on your laptop, free. The list grew to eleven messages — **req 5** re-sent all of them.

What Went Into req 1 — and Never Changed

This is T2's initialization, seen from the wire: those five steps exist to build exactly two arguments — then re-sent byte-identical on every later request.

	Assembled from	In this example
system	T2's init steps 1–4: global settings, CLAUDE.md, .claude/rules/*.md	"You are Claude Code. You have these tools..." + whatever house rules T4 had you write
tools	the schema of every tool the harness is willing to run	Bash, Read, Write, Grep... names, descriptions, argument types — <i>no code</i>

Yes — the model writes the code and asks your machine to run it. A Bash call is the model generating the *text* of a shell command as an argument. It executes nothing; your harness runs that string on your laptop, with your files and permissions.

The safety boundary is not the model's judgement — it is T2's permission dial, between the generated string and your shell.

system is the only text still guaranteed to be there on turn 200 — everything else drifts or compacts away. That is the mechanical reason CLAUDE.md earns its keep.

Reading the Trace: Four Things Just Happened

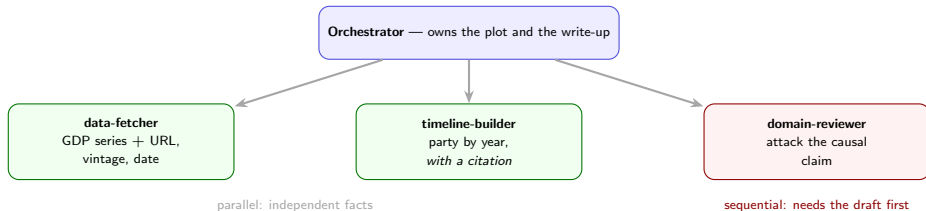
Every claim from §1, §3–§4 shows up in that one picture.

Moment	What it looked like	What it proves
The loop ran itself	five <code>tool_use</code> → <code>tool_result</code> round trips, no human input	nobody scheduled these steps; the model chose each one (§1)
The error was recovered	backend error came back as a <code>tool_result</code> with <code>is_error</code> , and the next turn retried	this is “Observe” in ReAct, and why a failed tool must still <i>return</i> (§1)
Cost grew quadratically	req 5 re-sent everything from req 1	the list is the cost driver, not the number of questions (§1)
One step had no tool	the party timeline was <i>written from memory</i> (orange, previous slide)	the weakest link: in the transcript it looks identical to a fetched fact

Where a referee would stop you. The GDP series has provenance — a URL, a vintage, a retrieval date. The party timeline has none; it is model recall typed into a CSV. **Ask of every number an agent hands you: which tool produced this, and can I re-run it?**

The Same Task with Three Agents

The useful question is not “is this faster” but which steps are independent, and which one needs an adversary.



What fan-out actually buys

- the two *facts* are independent → genuinely parallel
- 100 rows land in data-fetcher's context, **not** the orchestrator's
- one job each, so “cite your source” is enforceable

What it does not buy

- the plot is one small step — splitting it adds turns, not speed
- three agents cost more tokens for the *same* numbers
- nothing here makes the causal claim more true

Only domain-reviewer **changes the answer** — and it cannot run in parallel: it needs something to attack.

What the Reviewer Says — and What No Agent Can Fix

Given the draft, a competent domain-reviewer returns roughly this:

“The correlation is real but you cannot read it as an effect. Since 2000 you have about **six administrations** — your effective n is single digits, not the 100 quarters you plotted. Presidents inherit economies, business cycles are not assigned at random, and Congress is unmodelled. **Blinder and Watson (2016)** find a large gap over 1949–2012 (about **1.79pp** more annual real GDP growth under Democratic presidents, 16 terms) and attribute most of it to *oil shocks, productivity shocks, and foreign demand* — not to policy. Report the correlation, state the identification problem, and drop the word ‘affects’.” **[solid]**

This is the case for multi-agent, stated honestly. Not speed. A second agent whose job is to *disagree* caught a claim the first agent was happy to make — because you told it to.

And the limit. Every agent here inherited your word “*rationalize*”. A more agreeable reviewer would have obliged. **No number of agents fixes a leading prompt** — that is T5’s Context Dilemma, in one word of your own typing.

The fix is upstream and costs nothing: ask “*is there a relationship, and what would identify it?*” — then the same machinery works for you instead of against you.

The Corrections, in One Table

Everything here is checkable on your own machine — the vocabulary is marketing, the mechanism is not.

Widely repeated	What the machine does
"The session remembers me."	The API is stateless. A session is a list you re-send in full every turn — your cost curve and your context limit.
"A router dispatches specialists from a registry."	The same model, in the same context, emits a tool call. The registry is your folder of Markdown files.
"A skill is hard-coded software."	A skill is prose the model reads, disclosed progressively. Its description is what selection runs on.
"You can run unlimited agents."	Tokens-per-minute, context growth, and <i>your</i> coordination cost bind — usually the third.
"CLAUDE.md is just documentation."	It <i>is</i> the <code>system</code> argument of every request — built at initialization, and the only text still guaranteed to be there on turn 200.

One model, one loop, many lists.

Every lever you have — `system`, `tool`, `skill`, `sub-agent` — is prose you wrote.

Bridge: Now Design and Dispatch Your Own

The lab makes every claim in this deck falsifiable in about forty minutes.

Part A — the mechanism, offline

- Build a messages list by hand and watch it grow
- Run the §1 loop against a scripted model — no key, no network, no cost
- Plot tokens per turn: see the cost curve
- Wire the three patterns from §3 and measure them

Part B — dispatch, live in your terminal

- Write three agent files: code-reviewer, domain-reviewer, verifier — T4's RA team, made real
- Then write the fourth that team is missing: math-agent, from your filled canvas (§2)
- Dispatch them at a flawed Aiyagari solver
- Run the four checks from §4
- Narrow a description and watch dispatch stop
- See why *what an agent is allowed to read* decides what its review is worth
- Fill the 3×3 detection matrix and check the diagonal fires and the control row stays silent
- Then T2c: dispatch the same task at all three rungs, and script one